

Symbols

: (colon) 43
& (ampersand) 65
== operator 25
=> arrow 67
| (pipe symbol) 65, 275

A

actor
 implementing map2
 via 118–120
 overview 115–117
 using 117–118
ADTs (algebraic data
 types) 44–47
algebra
 algebraic reasoning and 96
 expressiveness 123
 parallel computations
 avoiding deadlock
 113–115
 law of forking 112
 law of mapping 110–112
 overview 110
 for parser combinators
 context-sensitivity 169
 failover 168–169
 implementing 166
 labeling parsers 167–168
 overview 147–153
 sequencing parsers
 166–167
algebraic data types. *See* ADTs
algebraic design 147, 152
algebraic reasoning 96, 110

ampersand (&) 65
anonymous function
 defined 24
 syntax for 8
 underscore notation 39–40
ANTLR library 147
applicative associativity
 law 215
applicative effects 209
applicative functors
 advantages of
 Applicative for
 streams 212
 overview 211
 validation with Either
 variant 212–214
applicative trait 206–208
laws
 associativity 215–216
 left and right
 identity 214–215
 naturality 216–218
monads vs.
 Option applicative vs.
 Option monad 209
 overview 208
 Parser applicative vs.
 Parser monad
 210–211
arrays, mutable 262–263
associative law
 Kleisli composition
 196–197
 overview 194–196
 proving 196–197
associativity 175, 215–216
asynchronous I/O 247–250

awkwardness in functional
 programming 83

B

backtracking 163
balanced fold 179–181
block 16
body 16
bottom 67

C

callback 115
category theory 187
checked exception 50
.class files 17
colon (:) 43
combinator
 information specified by 161
 monadic 193–194
 parsing 146
 refining to most general
 form 120–123
commit 163
companion object 32–33
composing monoids
 complex monoids 185–186
 overview 184
 using to fuse traversals 186
composition of processes 275
compositional style 293
compositionality 269
concurrency 98–99, 117
console I/O 243–246
context and referential
 transparency 264–266